

AI SYNTHESIS DOCUMENTATION

This document aims to identify and disclose the instances in which I used Claude (Anthropic) as an AI assistant during the development of the Python-based analysis and visualisation pipeline for this project. I would like to note that this assignment was completed prior to the class presentations, and as a result I was not fully aware of the documentation requirement at the time of development. Consequently, I do not have exact records of every prompt I used. However, to the best of my recollection, I have outlined below the key areas where AI assistance was involved, along with reconstructed descriptions of the problems I faced, the type of guidance I sought, and the outcomes that resulted. Each usage is presented on a separate page following the Before / The Prompt / After format.

TASK: Building the Thematic Map Visualisation (gorsellestir_q1.py)

BEFORE

I had a completed frequency table (frekans_tablosu.xlsx) and wanted to create a publication-quality thematic map that visually shows the hierarchy: research question → four themes → individual codes with frequency counts. I had the data and the theme structure but couldn't create boxes with arrows in matplotlib.

THE PROMPT

"I need to create a thematic map in Python using matplotlib. The map should show a research question box at the top, then four theme boxes below it connected by arrows, then code boxes under each theme. Each code box should show the code name and its total frequency (n=). Use FancyBboxPatch for rounded boxes and a greyscale colour scheme suitable for an academic journal."

AFTER

The AI produced the `chart_thematic_map()` function using `FancyBboxPatch` for rounded boxes, `annotate()` with `arrowstyle` for arrows, and a four-shade greyscale palette. The function reads frequency totals from the data dictionary and labels each code box as 'code (n=X)'. The output (figure1_thematic_map.png) was saved at 200 dpi.

TASK: Regex Extraction of Coded Segments (analiz.py)

BEFORE

I had manually tagged my interview transcripts using a custom notation: [codename]excerpt[/codename]. I needed a Python script to automatically extract all coded segments across three transcript files (coach_coded.txt, athlete_coded.txt, teacher_coded.txt), group them by participant group and code, and export a frequency table to Excel. I couldn't use regex for this kind of nested tag extraction.

THE PROMPT

"I have interview transcripts where I tagged segments with [code]text[/code] notation. Write a Python function that uses re.findall() to extract all code-excerpt pairs from a text file. Then group the results by participant group (Coach/Athlete/Teacher) and code label, count frequencies, and export a pivot table to Excel using pandas."

AFTER

The AI produced the kodlari_cikar() function using the pattern r'\\[(\\w+)\\](.*?)\\[/\\1\\]' with re.DOTALL to handle multi-line segments. It also generated frekans_tablosu_olustur() which creates a pandas DataFrame, applies groupby and unstack, adds a TOPLAM column, and exports to Excel. The script processed 88 total coded segments across 14 codes and 3 participant groups.

TASK: Grouped Bar Chart for Theme-by-Participant Comparison (gorsellestir_q1.py)

BEFORE

After creating the thematic map and frequency table, I needed a grouped bar chart that compares how each participant group (Coach, Athlete, Teacher) contributed to each of the four themes. I wanted the bars to use different hatching patterns instead of colours so the chart would remain readable in black-and-white printing. I also wanted data labels on top of each bar.

THE PROMPT

"Create a grouped bar chart in matplotlib that shows theme totals per participant group. Use numpy to position bars side by side. Use greyscale fill colours and add hatching patterns (///, ..., none) to distinguish groups without colour. Add value labels above each bar. Remove top and right spines and add a light horizontal grid."

AFTER

The AI produced `chart_academic_grouped_bar()` using numpy offsets to position three bar groups per theme. It applied hatch patterns and greyscale fills. A loop adds integer labels above each bar. Spines are hidden and a dashed yaxis grid is added. The output (`figure2_grouped_bar.png`) clearly distinguishes all three participant groups in monochrome.

